

Code-Layout, Readability and Reusability

Date	5 July 2026
Team ID	XXXXXX
Project Name	AI Learning Support Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing and indentation are maintained throughout the project.	Yes	The project follows consistent Python indentation (4 spaces) for readability.
2	Proper File Structure	Files and folders are logically organized.	Yes	Source code is organized into separate folders such as <code>services</code> , <code>models</code> , <code>data</code> , and the main <code>app.py</code> file.
3	Meaningful Variable Names	Variables clearly describe their purpose.	Yes	Variables such as <code>problem</code> , <code>field</code> , <code>bert_result</code> , <code>bilstm_result</code> , and <code>emotion_history</code> improve code readability.
4	Function / Method Names	Functions use descriptive names.	Yes	Functions like <code>predict()</code> , <code>detect()</code> , <code>build()</code> , <code>save()</code> , <code>load_history()</code> , and <code>get_gemini_response()</code> clearly

				indicate their responsibilities.
5	Code Comments	Inline and section comments explain the code.	Yes	Major sections in <code>app.py</code> and service modules include comments for easier understanding and maintenance.
6	Modular Design	Code is divided into reusable modules.	Yes	Emotion prediction, Gemini integration, analytics, history management, and logging are implemented as separate service modules.
7	No Redundant Code	Duplicate or unused code has been minimized.	Partial	Some UI rendering logic could be further refactored into reusable functions, but the application avoids major duplication.
8	Error Handling	Exceptions and runtime errors are handled appropriately.	Yes	The application handles model loading, empty user input, and Gemini API availability to prevent crashes during execution.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	<code>bilstm_model.py</code>	Python, TensorFlow	Emotion prediction	High
2	<code>bert_model.py</code>	Python, Hugging Face Transformers	Emotion prediction	High
3	<code>mixed_emotion.py</code>	Python	Mixed emotion detection	High
4	<code>prediction_schema.py</code>	Python	Standardizing prediction results	High
5	<code>gemini_service.py</code>	Python, Google Gemini API	Personalized learning guidance	High
6	<code>interaction_logger.py</code>	Python	Saving interaction history	High
7	<code>history_loader.py</code>	Python, Pandas	Loading previous interaction history	High